№6 ООП

План

- Наследование и полиморфизм
  - Наследование
    - Простое наследование
    - Множественное наследование
  - Полиморфизм

Наследование и полиморфизм

Наследование

Бывают два вида наследования:

- Простое
- Сложное

Простое наследование

```
class A {
     int a;
public:
     void show(){
          std::cout<<this->a;
     }
}

class B : public A {
     int b;
public:
}
```

У наследования есть проблема - возникают лишние наследованные поля, доступа к которым нет, но ресурсы они занимают.

При наследовании очень важно слово которым мы наследуемся. Таким образом наследуются поля:

- `publuc`
  - `privite` → `private`
  - `protected` → `protected`
  - `public` → `public`
- `protected`
  - `privite` → `private`
  - `protected` → `protected`
  - `public` → `protected`
- `private`
  - `privite` → `private`
  - `protected` → `private`
  - `public` → `private`

Множественное наследование

```
class A {
    int a;
public:
    void show(){
        std::cout<<this->a;
    }
}

class B : public A {
    int b;
public:
}

class C : public A {
```

```
}

class D : public B, public C {

}
```

При множественном наследовании проблема выделения лишней памяти усугубляется.

Полиморфизм

Полиморфизм - это разное поведение функции или класса в зависимости от использования.

Существует два вида полиморфизма:

- *Простой* - перегрузка операторов и методов, использование шаблонов.
- *Сложный* - возможность задания несколько реализаций единому по названию методу для классов разных уровней иерархий.

```
class A {
     int a;
public:
     virtual void show(){
          std::cout<<"sA";
     }
     void print(){
          show();
          std::cout<<"pA";
     }
}

class B : public A {
     int b;
public:
     void show() // тоже виртуальный так как наследуется
          std:cout<<"sB";
```

```
}

void action(A*a){
    a->show();
    a->print();
}

A*a=new.A();
A*b=new.B();

action(a); // sAsApA
action(b); // sBsBpA
```

Виртуальные методы добавляют к памяти переменных класса указатели на *виртуальную таблицу* в которой содержится адрес реализации метода. Когда *конструктор создаёт* экземпляр он переопределяет методы *с учётом* смещения из *виртуальной таблицы*. При этом для каждого класса виртуальная таблица своя. При этом виртуальные методы не могут иметь разные сигнатуры.

Виртуальный метод не имеющий реализации называется *абстрактным*. *Класс* которые содержит хотя бы один абстрактный метод называется *абстрактным*. Класс который имеет только абстрактные методы является *интерфейсом*.

`virtual void action()=0` - явное указание на абстрактность.